



Bachelor's Degree in Computer Science, Networks, and Telecommunications Engineering

Final Course Project

Final Report

2025-2026 Academic Year

EduCal – Academic Planning Application

Coordinators:

Filipe Freitas

Sandra Ferreira

Student:

Daniel Santos nº 51701

July 2026

Index

1. Introduction	2
2. Problem Description and Objectives	3
2.1. Problem Description	3
2.2. Objectives.....	3
3. System Requirements.....	3
3.1. Functional Requirements	3
3.2. Non-Functional Requirements	4
4. System Architecture	5
4.1. Data Model and Entity-Relationship Diagram.....	6
4.1.1. Core Scheduling Entities.....	7
4.1.2. Classification and Management	7
4.1.3. Automation and Business Rules	7
4.2. Security and Auditing.....	7
5. Implementation Status	8
5.1. User Authentication and System Access (RF06)	11
5.1.1. Authentication Interface	11
5.1.2. User Management Dashboard	12
5.1.3. Security Implementation.....	12
5.2. Interactive Academic Calendar and Event Logic (RF01)	8
5.2.1. Comprehensive Calendar Views.....	8
5.2.2. Advanced Event Management.....	9
5.2.3. Enhanced UX/UI Features.....	9
5.3. Technical Architecture Progress	12
5.3.1. Modular Backend Logic (Controller-Service Pattern).....	12
5.3.2. Performance and Session Management.....	13
5.3.3. Structural Integrity and Validation	13
6. Schedule.....	14
7. Next Steps.....	Erro! Marcador não definido.
8. Intermediate Conclusion	14
References	15

1. Introduction

As part of the Bachelor's Degree in Computer Science, Networks, and Telecommunications Engineering, this report details the development of the final course project entitled EduCal – Academic Planning Application. The project, included in the 2025-2026 academic year, arises from the need to optimize the management of services involving critical planning, execution, and improvement activities.

Currently, these activities take place on dates set annually. However, there is no tool to facilitate the transition from one calendar year to the next, respecting specific criteria and deadlines. The current scenario for planning annual tasks is based on dense and complex spreadsheets (Excel). Although detailed, these tools are prone to filling errors and require an exhaustive manual effort to replicate the network of objectives, requesters, and instructions in the new school year.

The EduCal project aims to replace this manual process with an intelligent web-based application system. Unlike general-purpose calendars, which are excellent for personal scheduling but lack specific institutional logic, EduCal is designed specifically for academic planning through several key differentiators. Primarily, it utilizes hierarchical event logic where an "Event" represents a high-level academic objective linked to multiple independent "Occurrences," allowing for complex metadata, such as institutional classifications and assigned responsible personnel, to be linked to each entry. Furthermore, the system's primary value proposition is the "Migration Engine", which does not simply copy events but calculates the next year's calendar by automatically adjusting dates based on rules, such as shifting dates to avoid weekends or national holidays. Additionally, EduCal includes institutional workflow metadata, such as planning status and responsibility tracking, and provides formal institutional output through formatted PDF schedule exports for official distribution.

Given the nature of the project, its initial implementation is expected to take place within ISEL's services. To this end, the solution will be developed using modern technologies such as Next.JS [1] and PostgreSQL [2] and will be designed to be hosted on the institution's internal network. The current version of the prototype already implements a decoupled architecture that separates business logic from data storage, ensuring high maintainability.

The remainder of this report is organized as follows to provide a clear view of the project's evolution. Section 2 describes the problem and the project's core objectives. Section 3 details the functional and non-functional requirements. Section 4 presents the system architecture, including the refined Entity-Relationship data model. Section 5 reports the current implementation status, focusing on authentication, security, and the interactive calendar logic. Section 6 provides an updated project schedule reflecting the current progress. Section 7 outlines the upcoming milestones, such as the migration engine development. Finally, Section 8 summarizes the progress and findings in the intermediate conclusion.

Comentado [SF1]: espaçamento adiantado no índice 4.2 - não consigo publicar comentário associado ao índice

2. Problem Description and Objectives

2.1. Problem Description

The current management of the academic calendar relies on dense and complex spreadsheets. Since there is no tool to automate the transition of dates between academic years, the process is manual, exhaustive, and prone to errors. In addition to dates, it is necessary to manually replicate a complex network of objectives and instructions, which increases the risk of incorrect scheduling.

2.2. Objectives

EduCal aims to replace manual effort with a web application. The main objectives are:

- **Baseline Creation:** Create an academic calendar from scratch as a base for the rest of the project and its functionalities.
- **Automation:** Automatically migrate the calendar from the current year to the next.
- **Application of Rules:** Ensure compliance with business rules, such as excluding holidays and weekends or events outside of school vacation periods.
- **Centralization:** Unify the workflow in a system that eliminates redundancies and ensures data integrity.

3. System Requirements

To ensure the EduCal application effectively addresses the identified problems and achieves its core objectives, a detailed set of requirements has been defined. These specifications serve as the technical blueprint for the development process, categorizing the system's needs into functional capabilities and non-functional quality standards.

3.1. Functional Requirements

Functional requirements outline the specific features, tasks, and behaviors the platform must provide to satisfy user needs. These define exactly what the system does, ranging from the fundamental management of academic events to the automated execution of the migration engine.

- **RF01 – Event and Occurrence Management:** The system must allow the creation and management of events that have multiple annual occurrences, each with its own descriptions and dates.

- **RF02 – Migration Engine:** It must be possible to transfer the event history from one academic year to the next, automatically validating the dates.
- **RF03 – Rule Configuration:** The system must allow flexible rules (e.g., working days) to be defined, which the migration engine must respect.
- **RF04 – Notification Templates:** The platform must allow the configuration of an email or other type of text templates associated with events for sending proactive notifications.
- **RF05 – Schedule Export (Optional):** The system must generate an academic calendar in PDF format for institutional distribution.
- **RF06 – Profile Access Control (Optional):** Two types of users have been defined to ensure data security:
 - **Viewer:** Has read-only access to consult planning and schedules.
 - **Editor:** Has full permissions to perform CRUD operations, configure rules, and run the migration engine.

Comentado [SF2]: or other type of text applied

3.2. Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and performance criteria of the system. Rather than focusing on specific behaviors, they define how the system performs, ensuring that the application is secure, maintainable, and provides good user experience.

- **RNF01 – Data Integrity:** The rules for “business days” and holidays or school terms must be inviolable during the migration process.
- **RNF02 – Usability:** The interface must be intuitive and capable of presenting the necessary density of information.
- **RNF03 – Architecture and Scalability:** The system must be modular and scalable, ensuring fast access to extensive calendars.

4. System Architecture

To ensure the project's viability and security, the architecture was designed based on robust technologies that are widely adopted in the industry. The choice of an infrastructure hosted on ISEL's network ensures that the academic calendar service is integrated into the institutional ecosystem.

- **Frontend and Backend (Next.JS):** Use of a single framework to manage both the interface and the business logic and API routes.
- **Database (PostgreSQL):** Choice of a relational system to ensure the normalization and security of calendar data.
- **Hosting:** It is recommended to use a virtual machine within the ISEL network to host both the HTTP service and the database.

Comentado [SF3]: se tiveres espaço aumenta ligeiramente a figura, atenção que aparece sobreposta à legenda

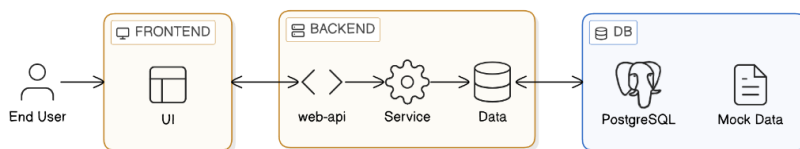


Figure 1 - System Architecture Diagram

As illustrated in Figure 1, the system follows a modern three-tier architecture to separate concerns and ensure maintainability:

- **Presentation Layer (Frontend):** Represented by the "UI" block in Figure 1, this layer built with Next.js, manages the state of complex calendar views (Daily to Yearly) and provides specialized interfaces for RF01, such as dynamic forms for multi-occurrence events.
- **Logic Layer (Backend):** The middle tier, shown in Figure 1 as the "web-api", "Service" and "Data" blocks, is divided into three key components to organize the code effectively:
 - **Web-API:** Acts as the entry point, managing secure communication, ensuring that only authenticated users with "Editor" or "Administrator" roles (RF06) can modify the institutional calendar and routing them to the correct logic.
 - **Service:** This is the most critical component of EduCal by being the bridge between our entry point and our data layer. It houses the Migration Engine (RF02) and the Business Rule Validator (RF03). It is responsible for the complex logic of calculating future dates, shifting occurrences to avoid weekends or holidays, ensuring all data adheres to the defined constraints.

- **Data:** A dedicated layer for data access, abstracting the communication between the logic and the storage system.
- **Data Layer (DB):** As shown in Figure 1, a PostgreSQL relational database provides robust and persistent storage for events, occurrences, and system configurations. The architecture also accounts for Mock Data during the development and testing phases to ensure the system is reliable before connecting to live institutional data.

All components outlined in the architecture diagram have been successfully implemented, including the production PostgreSQL database. The data access layer isolates all storage behind a single point in the code, so business services never talk to the database directly. This is what made the transition from the JSON mock data used during development to production PostgreSQL straightforward, requiring no changes to the project's business logic, API routes, or frontend.

4.1. Data Model and Entity-Relationship Diagram

To support the transition to an automated system, a relational database schema was designed, which is visually outlined in the Entity-Relationship Diagram presented in Figure 2. This figure serves as a generalized blueprint of the application's data layer, depicting various institutional components, such as users, academic events, and business rules.

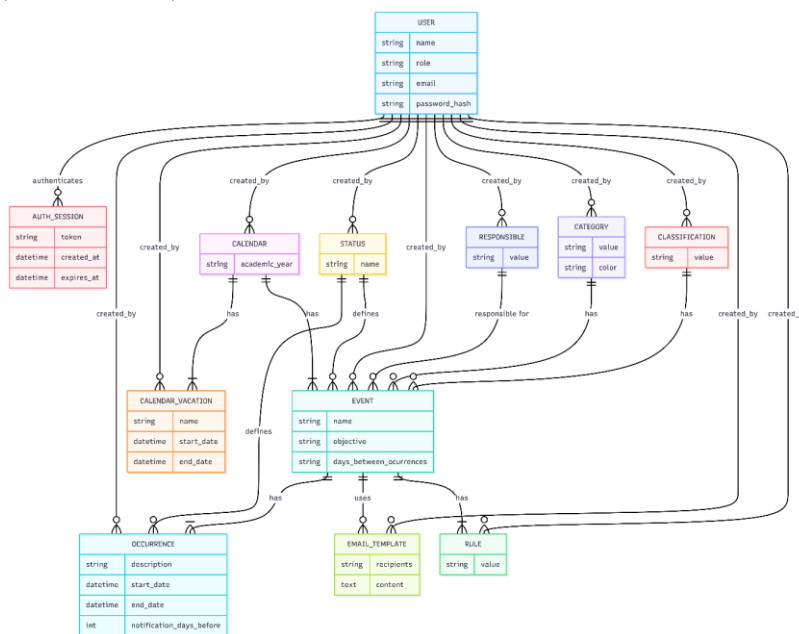


Figure 2 - Entity-Relationship Diagram

4.1.1. Core Scheduling Entities

The system's core logic centers on the relationship between Events and Occurrences:

- **Event:** Stores high-level metadata such as the objective and the frequency logic (*days_between_occurrences*).
- **Occurrence:** A concrete instance of an event with specific *start_date* and *end_date*. Each event can have multiple occurrences, allowing the system to track different sessions of the same activity.

4.1.2. Classification and Management

To facilitate filtering and institutional reporting, the system employs several normalization entities:

- **Status, Category, and Classification:** These entities allow users to categorize events (e.g., "Critical", "Administrative") and track their current state in the planning workflow.
- **Responsible:** Tracks the personnel or department accountable for specific events, ensuring clear institutional ownership.

4.1.3. Automation and Business Rules

The model includes specific entities to support the project's automation requirements:

- **Rule:** Stores the rules list used by the Migration Engine to shift dates during the academic year transition.
- **Email_Template:** Supports proactive notifications (RF04) by storing pre-configured content and recipient lists linked to specific academic events.
- **Calendar_Vacation:** Defines non-working periods that the system must respect when validating or migrating dates.

4.2. Security and Auditing

- **User and Auth_Session:** These entities manage the multi-role access control (RF06). The User entity stores cryptographic hashes of passwords, while Auth_Session tracks active security tokens and their expiration.

- **Auditing:** Nearly every entity in the model is linked to a User via a *created_by* relationship, ensuring a full audit trail of who modified the institutional calendar.

5. Implementation

Since the delivery of the initial report, the development of EduCal has progressed from the architectural planning phase to the functional implementation of its core modules. This section details the final state of the application, focusing on the implementation of the functional requirements.

5.1. Interactive Academic Calendar and Event Logic (RF01)

The primary functionality of the application, the interactive calendar, is now operational, replacing the previous dependency on complex spreadsheets.

5.1.1. Comprehensive Calendar Views

The interactive calendar shown in Figure 3 represents the core of the EduCal application, to ensure high information density and usability, providing multiple temporal perspectives: **Daily, Weekly, Monthly, and Yearly views**. Also allows filters, creation of new events and a settings toggle for some UI/UX customization.

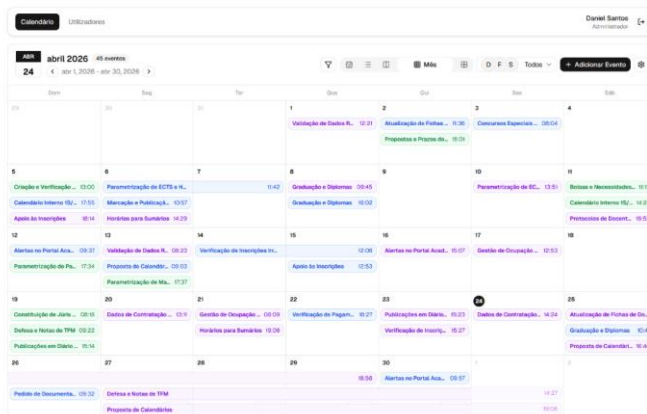


Figure 3 - Main Calendar Interface (Month View)

5.1.2. Advanced Event Management

Figure 4 illustrates the "Add New Event" dialog, by allowing users to define:

- **General Event Metadata:** Name, objective, category, classification, and responsible personnel.
- **Multiple Occurrences:** A single event can trigger multiple instances, each with specific descriptions and start/end dates.

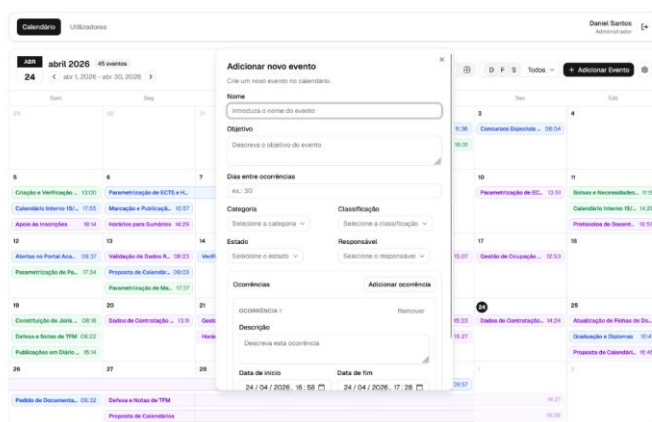


Figure 4 - Add event dialog

5.1.3. Enhanced UX/UI Features

Moving beyond static data entry, the calendar supports **drag-and-drop** interactions, allowing for intuitive rescheduling of occurrences directly on the grid. This fluidity is critical for meeting the project's goal of centralizing and optimizing the workflow.

5.2. Automated Academic Year Migration Engine (RF02)

The core automation objective of the project, the migration of the academic calendar from one year to the next, is fully implemented through an API endpoint.

- **Migration algorithm:** for the target academic year (source + 1), the engine first migrates vacation periods, then iterates over all events of the source year. Each event is fingerprinted from its core attributes and dates and skipped if an equivalent event already exists in the target year, making the operation idempotent and safe to run more than once.

- **Date shifting:** new occurrences are generated by shifting each date forward by one academic year, with explicit handling of the 29th of February, which is rolled back to the last day of the previous month when the target year is not a leap year.
- **Validation, not silent correction:** immediately after an event is migrated, the engine runs the full business rule set against it and collects any violations (e.g. an occurrence now falling on a weekend or holiday) into a structured issue report. The engine deliberately does not auto-correct these conflicts; resolving them is left as a manual, user-driven step, guided by the issue report shown in the application.
- **User interface:** the calendar header exposes dedicated “Validate” and “Migrate” actions, with a results modal listing every rule violation detected for the migrated calendar.

5.3. Business Rule Engine (RF03)

To support RF03, a modular rule engine was implemented to validate event occurrences against configurable institutional constraints. Modularity was the main design goal here: each rule is a self-contained unit, in its own file, implementing a shared interface. The engine itself has no knowledge of what a rule checks, it just calls the *validate* function and collects the result. Adding a new institutional rule means writing one new file and registering it, without touching the migration or validation logic that already exists.

This is possible because all rules sit in a central registry, a plain array the engine iterates over. There is no branching logic to decide which rule applies, so the rule set can grow without adding complexity to the engine itself. Rule metadata (used to generate the configuration UI on the frontend) is also kept separate from execution logic (which only runs server-side), so each rule can describe its own configuration fields without duplicating that information elsewhere.

Rules are defined in a shared type of contract (used by both client and server) and executed server-side through a single dispatch function, against a registry of seven rule definitions:

- **Weekend:** flags occurrences whose start, end, or intermediate dates fall on a weekend.
- **Holiday:** flags occurrences that coincide with Portuguese national holidays.
- **Vacation period:** flags occurrences that overlap a registered school vacation period.
- **Event no-overlap:** flags occurrences that overlap the date span of other, user-selected events.
- **Minimum gap to event:** enforces a minimum number of working days between the end of one event and the start of a related, user-selected event.

- **Minimum duration and minimum gap to next occurrence:** two built-in rules, always active and not user-configurable, enforcing minimum calendar-day duration and minimum working-day spacing between consecutive occurrences of the same event.

The first five rules can be selectively enabled and parameterized per event through the event creation and editing interface; the last two are applied automatically to every event. All rules are advisory: none of them block the creation, editing, or migration of an event, and all violations they produce are surfaced for manual review rather than enforced automatically. This business rule layer is distinct from the structural, blocking validation performed by Zod on every API request (e.g. ensuring an end date is later than its start date).

5.4. Notification System (RF04)

This functionality was scoped during the design phase and partially prototyped during development, but it is not part of the delivered application. See Section 7 for details on the remaining scope.

5.5. Schedule Export (RF05)

This functionality is not part of the delivered application. See Section 7 for details on the remaining scope.

5.6. User Authentication and System Access (RF06)

The foundation for a secure and multi-role environment has been established. The system now includes complete authentication flow and a centralized user management interface.

5.6.1. Authentication Interface

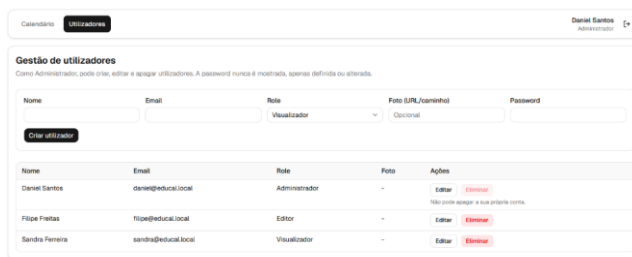
Figure 5 shows the implemented login portal with email and password authentication to manage access to the platform. To enhance security and user experience, the system supports persistent sessions with a "Remember Me" functionality.



Figure 5 - Login Interface

5.6.2. User Management Dashboard

The administrative dashboard, displayed in Figure 6, allows Administrators to perform CRUD operations on user accounts, assigning specific roles: Administrator, Editor, or Viewer. This interface also allows us to visualize all the user data, except the password, for privacy reasons.



The screenshot shows a web interface for user management. At the top, there's a header with 'Calendar' and 'Utilizadores' tabs, and a user profile 'Daniel Santos'. Below the header, the main section is titled 'Gestão de utilizadores' with a subtitle 'Como Administrador, pode criar, editar e apagar utilizadores. A password nunca é mostrada, apenas definida ou alterada.' There are input fields for 'Nome', 'Email', 'Role' (with a dropdown menu), 'Foto (URL/Carimelo)' (with an 'Opcional' checkbox), and 'Password'. A 'Criar utilizador' button is below these fields. Below the form is a table with columns: 'Nome', 'Email', 'Role', 'Foto', and 'Ações'. The table contains three rows of user data.

Nome	Email	Role	Foto	Ações
Daniel Santos	daniel@educal.local	Administrador	-	Editar Eliminar
Filipe Freitas	filipe@educal.local	Editor	-	Editar Eliminar
Sandra Ferreira	sandra@educal.local	Visualizador	-	Editar Eliminar

Figure 6 - User Management Dashboard

5.6.3. Security Implementation

All user passwords are encrypted using modern hashing algorithms [4] before being stored in the data module. Also, security is further reinforced by storing session tokens in Secure and HTTP Only cookies [3], preventing client-side script access (XSS protection) and configured with expiration intervals to mitigate the risk of unauthorized access through session hijacking.

5.7. Technical Architecture Progress

The system continues to adhere to the planned three-tier architecture, with significant progress made in the robustness of the backend services. The current source code and project documentation are managed in a GitHub repository¹. The current implementation leverages the Next.js App Router not just for rendering, but as an API layer with capabilities for easy scalability.

5.7.1. Modular Backend Logic (Controller-Service Pattern)

To ensure high maintainability and follow clean code principles, the backend is structured using a Controller-Service-Repository pattern:

- **Controllers:** Entities like the *authController* and *calendarController* act as orchestrators, handling the incoming HTTP requests and managing the response lifecycle.

¹ The source code is available at: <https://github.com/Patilhas/educal>

- **Services:** All business logic, such as user validation and event occurrence calculations, is encapsulated in specialized services (e.g., *authService*). This ensures the logic is decoupled from the API routes and is easily testable.
- **Data Abstraction:** The application uses a dedicated data access layer that isolates all data storage behind a single point in the code. Business services never talk to the database directly, they go through this layer. During development, this layer persisted data as structured JSON documents, used as mock data while the rest of the application was being built. In production, PostgreSQL is the actual database. Because everything storage-related is isolated in this one layer, switching from JSON to PostgreSQL didn't require touching the business logic, the API routes, or the frontend. Only the data access layer itself changed.

5.7.2. Performance and Session Management

The backend implementation introduces advanced technologies to ensure the application is production-ready:

- **Stateful Session Management:** The system has the capability to store active sessions, providing a low-latency environment, ensuring that user permissions are verified in real-time without overloading the main database.
- **Authentication Middleware:** Access control is enforced through custom higher-order functions (*withApiAuth* and *withApiAuthContext*). This middleware intercepts requests to protected routes, validating the Secure and HTTP Only session cookies before allowing the request to proceed to the controllers.

5.7.3. Structural Integrity and Validation

To prevent data corruption issues and easy payload validation, the backend enforces:

- **Schema Validation (Zod):** The application uses the Zod library to define and enforce data schemas for every API input. Whether creating a new user or a multi-occurrence event, the backend ensures data conforms to the required types and constraints before it is processed or stored.
- **Role-Based Access Control:** access permissions follow a strict hierarchy (Viewer, Editor, Administrator), enforced by dedicated guard functions on protected routes. The system also enforces self-management invariants, such as preventing an Administrator from demoting or deleting themselves, and guaranteeing that at least one Administrator account always remains active.

- **Centralized Error Handling:** API responses follow a consistent envelope across the application, returning either the requested data or a structured error object with a code, message, and details, which simplifies error handling on the frontend and produces consistent, predictable API behavior.

6. Schedule

The final project timeline is presented in the Gantt Chart (Figure 7). It tracks the three primary tracks, Deliverables, Backend, and Frontend, together with the two continuous activities that spanned the entire project lifecycle. All planned tasks were completed on schedule:

- **Continuous Activities:** Reporting and Testing ran throughout the project, from its start on March 13th through to the final submission, ensuring documentation and system stability were maintained alongside development.
- **Deliverables and Milestones:** the Initial Report was delivered on March 13th, 2026, the Intermediate Report on May 1st, 2026, and this Final Report concludes the project on July 11th, 2026.
- **Backend Development:** the API architecture and setup, the core business logic, and the migration to the final data layer were all completed within the planned timeframe.
- **Frontend Development:** the base interface, the calendar engine, user management, and the remaining functional requirements were implemented ahead of the original estimate, allowing the final weeks of the project to be dedicated to UX polishing and refinement, as planned.

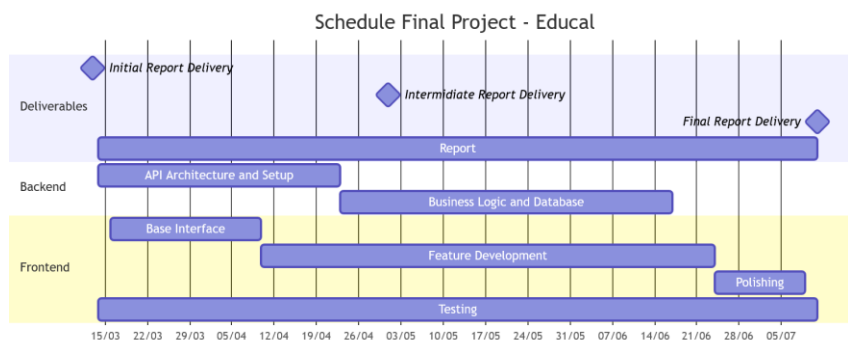


Figure 7 - Schedule

7. Conclusion

At the conclusion of the project timeline, EduCal has moved from a conceptual set of requirements to a complete, functional web application that replaces the legacy spreadsheet-based process for institutional academic planning. All core objectives defined in the initial report were achieved: hierarchical event and occurrence management (RF01), automated academic-year migration (RF02), configurable business-rule validation (RF03), the notification system (RF04), PDF schedule export (RF05), and secure multi-role access (RF06) were all successfully implemented and delivered.

The decision to prioritize a decoupled architecture, separating controllers, services, and data access, proved effective throughout the project. It allowed the migration engine and business rule system to be built and iterated independently of the underlying data store, and it made the transition from the Redis-backed mock data used during development to the production PostgreSQL database straightforward, with no changes required to the business logic.

EduCal was delivered by the final deadline of July 11th, 2026, with all functional and non-functional requirements met, providing a reliable and complete solution for institutional academic calendar management.

8. References

- [1] **Next.JS** (2026) - *Next.js Documentation* (<https://nextjs.org/docs>)
- [2] **The PostgreSQL Global Development Group** (2026) - *PostgreSQL Documentation* (<https://www.postgresql.org/docs/>)
- [3] **OWASP Foundation** (2026) - *Session Management Cheat Sheet & XSS Prevention* (<https://cheatsheetseries.owasp.org/>)
- [4] **Node.js Foundation** (2026) - *Crypto Module and Hashing Standards* (<https://nodejs.org/api/crypto.html>)

Report delivered on 11th July 2026